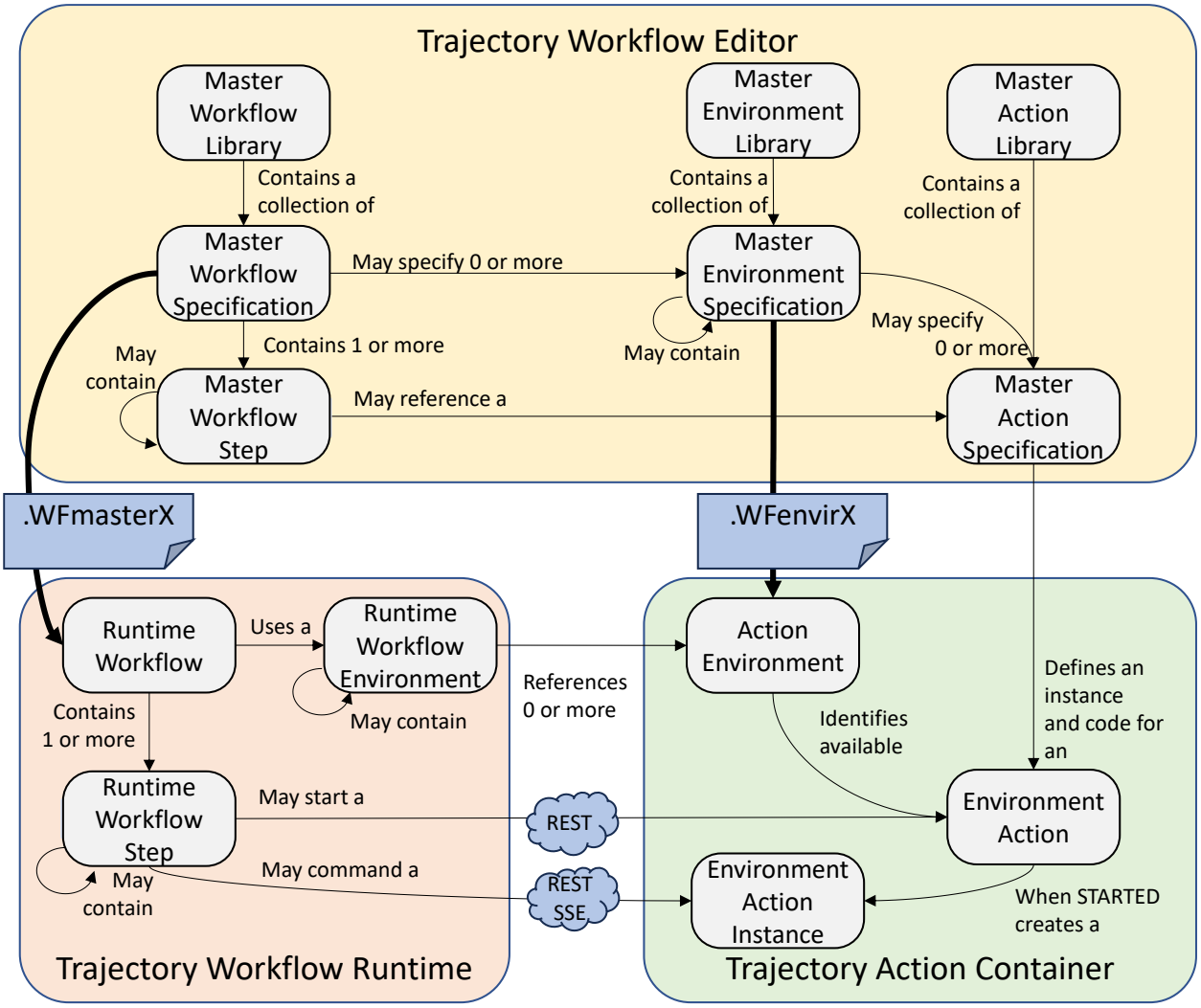
Trajectory Workflow Help Guide

PLEASE NOTE: Trajectory is a demonstration system, not intended for production environments. The editor and runtime are single-user systems and do not have the security necessary for production use. We recommend loading the applications into a Docker container for your testing.

The three main Trajectory elements are the **Trajectory Workflow Editor**, the **Trajectory Runtime** (with a WEB version and an Android version), the **Trajectory Action Container** for python code. There is also the **Trajectory Action Tester**, which can be used to test any action container.


Trajectory Workflow Editor
Master Workflow Library
Contains a collection of
Master Workflow Specification
May specify 0 or more
Contains 1 or more
May contain
Master Workflow Step
May reference a
Master Environment Library
Contains a collection of
Master Environment Specification
May contain
May specify 0 or more
Master Action Library
Contains a collection of
Master Action Specification
.WFmasterX
.WFenvirX
Runtime Workflow
Uses a
Runtime Workflow Environment
May contain
Contains 1 or more
Runtime Workflow Step
May start a
May command a
May contain
Trajectory Workflow Runtime
References 0 or more
REST
REST SSE
Action Environment
Identifies available
Defines an instance and code for an
Environment Action
Environment Action Instance
When STARTED creates a
Trajectory Action Container

Trajectory Workflow Guide

Contents

1 Trajectory Workflow Editor 7
1.1 Getting Started 7
1.1.1 Signing In 7
1.1.2 Creating Your First Workflow 7
1.2 Screen Layout 8
1.2.1 Library Sidebar 9
1.2.2 Searching 9
1.2.3 Managing Libraries 9
1.2.4 Dragging Specifications to Canvas 9
1.2.5 Export and Import Controls 9
1.2.6 Adding Steps to the Canvas 10
1.2.7 Canvas 11
1.2.8 Properties Panel 13
1.2.9 Toolbar 16
1.2.10 Canvas Toolbar (Top-Right of Canvas) 16
1.3 Node Types Reference 17
1.3.1 Start 17
1.3.2 End 17
1.3.3 Environment Action 17
1.3.4 User Interaction Step 17
1.3.5 Yes/No Step 17
1.3.6 The User Interface Canvas 18
1.4 Script Step 19
1.4.1 Input Parameters 19
1.4.2 Output Parameters 20
1.4.3 Example 20
1.5 Child Workflow 20
1.6 Select 1 20
1.7 Parallel / Wait All 21

1.8 Wait Any 21
1.9 Catch 21
1.10 Return 21
1.11 Notation Systems 21
1.11.1 Flowchart Style (Default) 22
1.11.2 BPMN Style 22
1.11.3 ISA 88 Style (Procedural Function Chart) 23
1.11.4 Switching Notations 23
1.12 Environment and Action System 23
1.12.1 Concept 23
1.12.2 State Models 24
1.12.3 Linking Environments to a Workflow 25
1.12.4 Using Actions in a Workflow 25
1.12.5 Creating Action and Environment Specifications 25
1.12.6 Registered Action Servers 26
1.13 TRY, CATCH, and RETURN 26
1.14 Resources 27
1.14.1 Resource Types 27
1.14.2 Resources Scope 27
1.14.3 Resource Definition 28
1.14.4 Resource Commands 28
1.14.5 Execution Order 29
1.14.6 Resource Scope and Inheritance 29
1.15 Child Workflows 29
1.15.1 Creating a Child Workflow 29
1.15.2 Navigating Child Workflows 30
1.15.3 Embedded vs. Referenced 30
1.16 Export and Import Formats 30
1.16.1 Exporting 31
1.16.2 Importing 31

1.16.3 What's Inside a Workflow Package (.WFmasterX) 31
1.16.4 What's Inside a Library Package (.WFlibX) 32
1.16.5 What's Inside a Single-Environment Package (.WFenvirX) 32
1.16.6 Schema Version 32
1.16.7 Identifiers 32
1.17 Version and State Management 32
1.17.1 Versions 32
1.17.2 Lifecycle States 33
1.18 Other User Interaction 33
1.18.1 Saving 33
1.18.2 Editing Keyboard Shortcuts 33
1.18.3 Selection Buttons 33
1.18.4 Draft Recovery 34
1.18.5 Recovery on App Load 34
1.18.6 When Drafts Are Cleared 34
1.19 User Accounts and Roles 34
1.19.1 Role System 34
1.19.2 Ownership and Sharing 35
1.19.3 Account Management 35
2 Trajectory Runtime 36
2.1 Executing a Workflow 36
2.2 Export the workflow package 36
2.3 Run it on an Android phone or tablet 36
2.4 Run it in the web runtime (Trajectory Runtime) 37
2.4.1 Loading a Workflow 39
2.4.2 Running a Workflow 39
2.4.3 Environment Actions and Action Servers 40
2.4.4 Overview and History 40
2.4.5 Settings 40
3 Trajectory Action Container 42

3.1 Action Getting Started 42
3.2 Environments and Actions 42
3.3 Writing Action Code 43
3.3.1 Step and Action State Model 43
3.3.2 Coordinating state transitions from code 44
3.3.3 Code Examples 44
3.4 Testing Code 46
3.5 Instances 46
3.6 Execution Log 46
3.7 Settings 46
3.8 Connecting a Workflow to the Container 47

1 Trajectory Workflow Editor

The Trajectory Workflow Editor is a visual workflow specification editor for designing step-by-step procedures, recipes, and batch processes. It produces portable specification files consumed by The Trajectory Runtime and Action Containers.

1.1 Getting Started

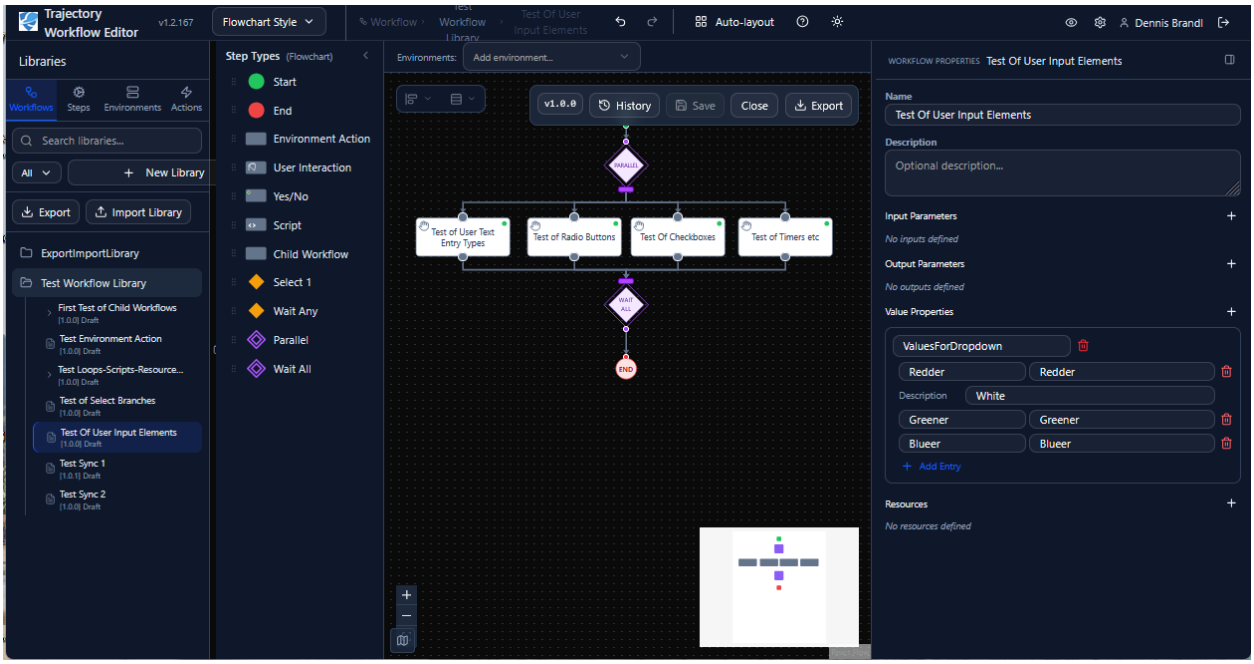
1.1.1 Signing In

1. Open Trajectory Workflow Editor in your browser.
2. Enter your email and password (minimum 8 characters).
3. If you are the first user, your account is automatically promoted to Administrator.
4. New users can register from the login page by clicking "Don't have an account? Register".

1.1.2 Creating Your First Workflow

1. Sign in to reach the main editor.
2. In the Library Sidebar on the left, select the **Workflows** tab.
3. Click **Create Library** at the bottom to create a workflow library.
4. Enter a name and click **Create**.
5. Expand the library and click **Add Specification** (+ button) to create a workflow specification.
6. The canvas appears in the center. Drag steps from the Step Palette onto the canvas.
7. Connect steps by dragging from one node's handle to another.
8. Your work is saved automatically.

1.2 Screen Layout



The editor has four main areas:

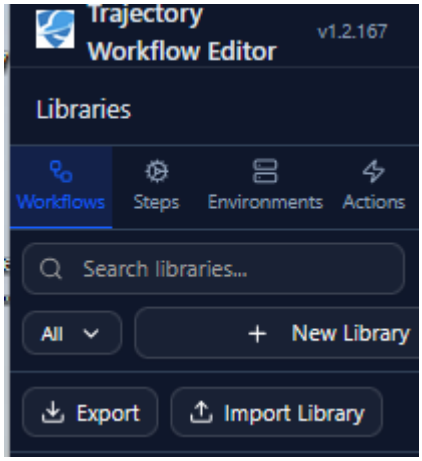
```
+-------------------------------------------------------------------------------+
| Toolbar: Trajectory | Notation | Back | Breadcrumb | Undo Redo | Layout | User  |
+----------+--------+-----------------------------------------------+------------+
|          |        |                                               |            |
| Library  | Step   |         Canvas                                | Properties |
| Sidebar  |Palette |                                               | Panel      |
|          |        |                                               |            |
|          |        |                                               |            |
+----------+--------+-----------------------------------------------+------------+
```

- Toolbar (top) -- Notation switcher, navigation, undo/redo, auto-layout, user menu
- Library Sidebar (far left) -- Browse and manage libraries and specifications
- Step Palette (left of canvas) -- Drag step types onto the canvas
- Canvas (center) -- Visual workflow editor with drag-and-drop nodes and connections
- Properties Panel (right) -- View and edit properties of the selected node or workflow

When viewing Environment or Action specifications, the canvas area is replaced by a form editor.

1.2.1 Library Sidebar

Four tabs at the top of the sidebar switch between library types:



Tab	Icon	Content
Workflows	Workflow	Workflow libraries and specifications
Steps	Gear	Reusable step library templates
Environments	Server	Environment definitions with action bindings
Actions	Lighting	Action specifications with inputs/outputs

1.2.2 Searching

Type in the search box to filter libraries and specifications by name. The search is case-insensitive and matches against both library and specification names.

1.2.3 Managing Libraries

Create a library: Click the **Create Library** button at the bottom of the sidebar.

Library context menu: (three-dot button on hover):

- Add Specification -- Create a new specification within the library
- Add Sub-Library -- Create a nested child library
- Rename -- Change the library name
- Delete -- Remove the library and all its specifications (cannot be undone)
- Share Library / Make Private -- Toggle visibility for other users (admin only)
- Import Workflow Package -- Import a .WFmasterX runtime package (workflow libraries only)

Delete a specification: Hover over the specification name and click the trash icon.

1.2.4 Dragging Specifications to Canvas

In the Workflows tab, you can drag a specification from the sidebar onto the canvas to create a pre-configured node with the specification's inputs, outputs, and values.

1.2.5 Export and Import Controls

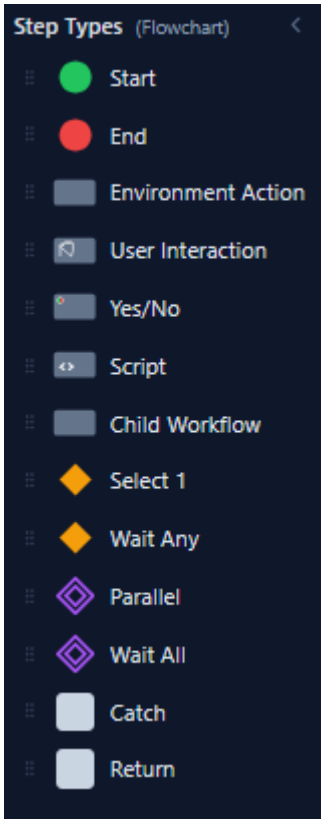
Below the search box:

- Export menu (download icon) -- Export the selected library
- Import button (upload icon) -- Import a library from file

- Step Palette

The Step Palette appears to the left of the canvas when editing a workflow. It shows available step types as draggable buttons. The header displays the current notation style (e.g., "BPMN", "ISA 88", and “Flowchart").

1.2.6 Adding Steps to the Canvas



Click and drag any step type from the palette onto the canvas. The node snaps to a 10-pixel grid and centers on the drop point.

1.2.6.1 Step Types - Control Flow

STEP	PURPOSE
Start	Entry point of the workflow
End	Exit point of the workflow
Select 1	Conditional branching, routes to one of several options based on a comparison condition.
Wait Any	Waits for any one of multiple parallel paths to complete
Parallel	Splits execution into parallel branches
Wait All	Waits for all parallel branches to complete
Catch	Defines the start of a CATCH workflow to catch errors, timeouts, and aborts of Environment Actions.
Return	Defines the end of a CATCH workflow with options to Abandon the workflow, restart the workflow, go to a step, or retry the trigger step.

1.2.6.2 User Interface Steps

STEP	PURPOSE
User Interaction	Design a runtime screen with the UI Canvas -- combine text, images, video, text inputs, multi-line text, checkbox and radio groups, dividers, and timers
Yes/No	A User Interaction screen plus a built-in two-way Yes/No branch

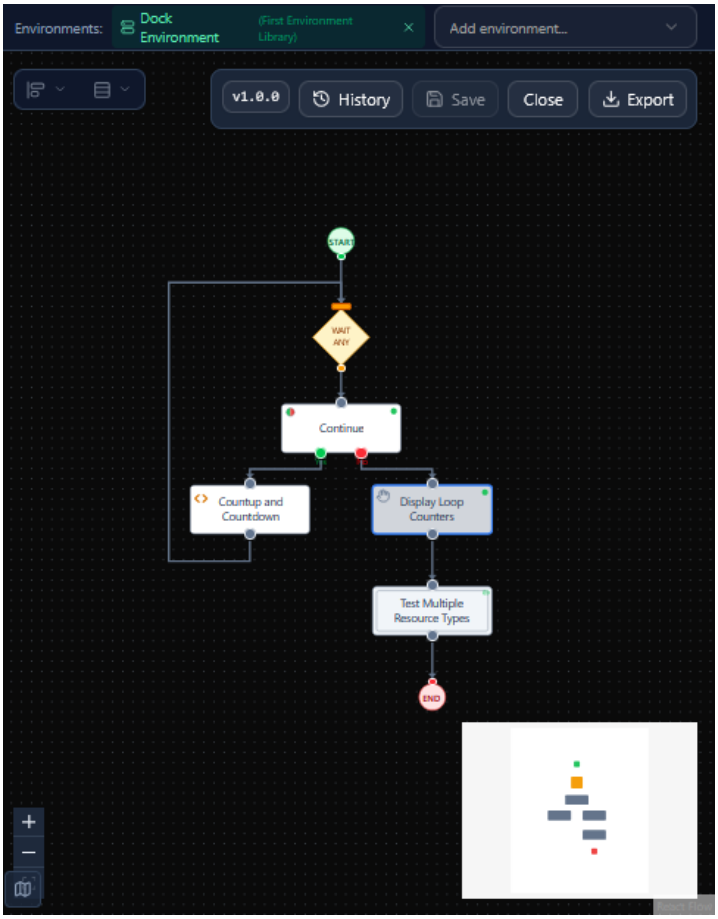
1.2.6.3 Execution Steps

STEP	PURPOSE
Script	Execute TypeScript code with input/output parameter access
Child Workflow	Embed or reference a sub-workflow

1.2.6.4 Environment Actions (dynamic)

Dragging an environment button onto the canvas creates an action node pre-linked to that environment.

1.2.7 Canvas



1.2.7.1 Navigating the Canvas

Action	How
Pan	Left-click or middle-click and drag on empty canvas
Zoom	Scroll the mouse wheel (range: 0.1x to 2x)
Fit view	Click the fit-view button in the bottom-left Controls widget
Jump to area	Click in the MiniMap (bottom-right corner)

1.2.7.2 Selecting Nodes and Edges

Action	How
Select One	Click a node or edge
Multi-Select	Hold **Shift** or **Ctrl** and click additional items
Box Select	Click and drag on empty canvas to draw a selection box

Deselect All	Click on empty canvas

Box selection uses partial mode: nodes partially inside the selection box are included.

1.2.7.3 Creating Connections

1. Hover over a node to reveal its connection handles (small circles).
2. Click and drag from a source handle to a target handle on another node.
3. Release to create the connection.

1.2.7.4 Connection rules:

- START nodes can only be a connection source (no incoming connections)
- END nodes can only be a connection target (no outgoing connections)
- Self-connections are not allowed
- Duplicate connections between the same handles are not allowed
- Connection snap radius is 10 pixels

Existing connections can be reconnected by dragging their endpoints to a different node.

1.2.7.5 Double-Click Actions

Target	Action
Node	Opens the Node Properties Dialog for editing
Embedded Child Workflow Node	Navigates into the child workflow canvas if it is not a reference copy. Reference copies can only be edited from the Step Library.
Empty Canvas	Opens the Workflow Properties Dialog

1.2.7.6 Deleting Nodes

Select one or more nodes and press **Delete** or **Backspace**. If any selected node has configured parameters, options, or settings, a confirmation dialog appears:

> *"Delete selected step(s)? This will remove configured parameters, options, and settings. (Use Ctrl+Z to undo if needed)"*

1.2.7.7 MiniMap

The MiniMap in the bottom-right corner provides an overview of the entire workflow. Nodes are color-coded:

- Green -- Start nodes
- Red -- End nodes

- Amber -- Select 1, Wait Any
- Purple -- Parallel, Wait All
- Gray -- All other nodes

You can pan and zoom within the MiniMap to navigate quickly.

1.2.8 Properties Panel

The Properties Panel on the right side shows context-sensitive information based on what is selected on the canvas.

1.2.8.1 No Selection (Workflow Summary)

When nothing is selected, the panel displays a summary of the current workflow:

- Workflow name
- Input parameters
- Output parameters
- Value properties
- Resources

1.2.8.2 Single Node Selected

Shows the node's details:

- Type badge (e.g., "ENVIRONMENT ACTION:", "USER INTERACTION:")
- Label
- Inputs and outputs (for parameterized types)
- Value properties
- UI parameters (for User Interaction and Yes/No steps)
- Resource commands

1.2.8.3 Single Edge Selected

For edges from *Select 1* nodes: shows the Edge Condition Editor with a condition expression field.

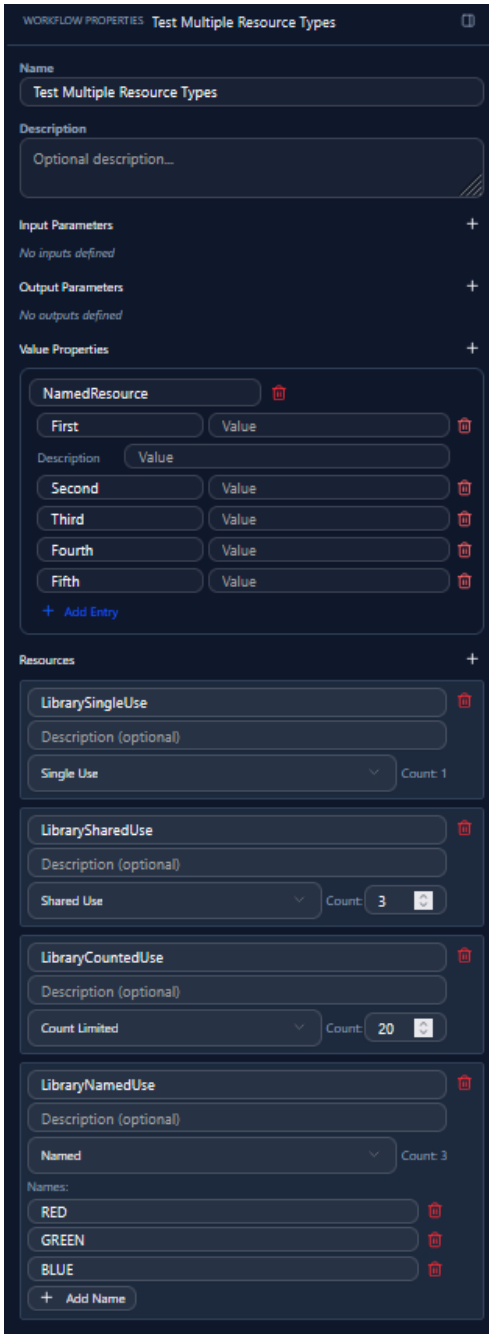
For edges from other node types: displays a message that condition expressions are only available for Select 1 connections.

1.2.8.4 Multiple Selection

Shows the count of selected items and a message to select a single item for editing.

1.2.8.5 Input Parameters

Each input parameter has a name, an optional description, and a value. The value mode is toggled via *Text* / *Source* radio buttons:



- **Text** -- A literal string value typed directly into the field.
- **Source** -- A reference to a Value Property, parameter, or step output. A dropdown picker lists available values:
  - *Value Property names* (e.g., `Temperature`) -- references the entire property
  - *Value Property entries* (e.g., `Temperature.Value`) -- references a specific entry within the property, shown indented below the property name
  - *Workflow Input Parameters* -- the enclosing workflow's input parameters
  - *Workflow Output Parameters* -- the enclosing workflow's output parameters
  - *Parent Output Parameters* -- the parent workflow's output parameters (when editing a child workflow)
  - *Parent Input Parameters* -- the parent workflow's input parameters (when editing a child workflow)
  - *Step Output Parameters* -- output parameters from other steps in the workflow, grouped by step name

  NOTE: When selecting an output parameter from another step, it should be from a preceding step in the workflow execution order. The editor does not enforce execution order -- it is the author's responsibility to ensure the source step runs before the consuming step.

1.2.8.6 How Property Values Are Passed to a Step

When an input parameter has its value mode set to **Source** (`value_type: "property"`):

- Referencing a single entry** (e.g., `Temperature.Value`) -- Only that entry's value is passed as a plain string (e.g., `"22"`).
- Referencing an entire *Value Property* (e.g., `Temperature`) -- The entire property is passed as a JSON array of name/value pairs:

````
```json
[
  { "name": "Temperature", "value": "22" },
  { "name": "Description", "value": "The Average Temperature over 1 hour" },
  { "name": "Units", "value": "Deg C" }
]
```
````

1.2.8.7 Output Parameters

Each output parameter has a *name* and a *value destination* selector. The destination determines where the step's output value is stored after execution. The dropdown lists available destinations:

- Value Property entries (e.g., `Temperature.Value`) -- stores the output in a specific Value Property entry, grouped by workflow level
- Workflow Output Parameters -- the enclosing workflow's own output parameters
- Workflow Input Parameters -- the enclosing workflow's own input parameters
- Parent Output Parameters -- the parent workflow's output parameters (when editing a child workflow)
- Parent Input Parameters -- the parent workflow's input parameters (when editing a child workflow)
- Add Value Property... -- creates a new Value Property inline and selects its Value entry
- Add Entry to Property... -- adds a custom entry to an existing Value Property

1.2.8.8 Value Properties

Each Value Property has a name and a list of key/value entries. New properties are created with two default entries: **Value** and **Description**.

- The *Value* entry key is editable -- rename it to match your data model (e.g., `SetPoint`, `Reading`).
- The *Description* entry key is read-only and cannot be deleted and is not passed to Environment Actions.
- Additional entries can be added with the *Add Entry* button

- Entries can be removed with the trash icon.
- Entry key names cannot contain dots or spaces (reserved for dot-notation references).

Value Properties may be defined in a workflow, or in a child workflow, in which case they are available to the workflow, or all child workflows. Value properties provide one method to share information across workflow steps. The other method is through the linking of input parameters of a steop to the output parameters of a previous step.

Value properties may also be defined in an Environment, in which case that are available to workflows that are using the environment. Environment Value Properties provide a method to share information across workflows. The other method is to the SYNC SEND and SYNC RECEIVE to exchange value property across workflows.

1.2.9 Toolbar

The toolbar at the top of the screen provides the following controls:

Control	Description
Trajectory	Application title
Notation Switcher	Dropdown to switch between Flowchart, BPMN, and ISA 88 notation styles
Back	Appears when editing a child workflow; returns to the parent workflow
Context Breadcrumb	Shows the current library and specification path
Undo	Undo the last change (up to 100 steps)
Redo	Redo a previously undone change
Auto-layout	Automatically arranges all nodes using the dagre graph layout algorithm |
User icon + name	Shows the current user's identity
Account Management	Manage user accounts (admin only
Sign Out	End the current session

1.2.10Canvas Toolbar (Top-Right of Canvas)

Button	Description
Export Workflow Package	Creates a self-contained runtime package (.WFmasterX) containing the workflow, all dependent environment and action libraries, and referenced images

1.3 Node Types Reference

1.3.1 Start

Entry point for the workflow. Only one Start node is recommended per workflow. Cannot receive incoming connections.

1.3.2 End

Exit point for the workflow. You can have multiple End nodes. Cannot have outgoing connections.

1.3.3 Environment Action

Executes an ACTION available from a linked environment. See the Trajectory Action Container for how to create and manage Environment Actions written in Python.

When editing:

- Select an action from the **Action Reference** dropdown (grouped by environment)
- Inputs and outputs are auto-populated from the action specification (read-only when an action is selected)
- Visibility can be "opaque" or "observable"

1.3.4 User Interaction Step

Displays a screen to the operator at runtime and (optionally) captures input. Double-click the step to open the **UI Canvas**, where you lay out the screen by placing and arranging elements. The node label on the canvas derives from the first Text element on the screen.

NOTE: The Android runtime uses the native Android services to position the UI elements. The editor only provides relative positioning.

1.3.5 Yes/No Step

A User Interaction screen **plus** a built-in two-way branch. It uses the same UI Canvas with the same set of elements, so a Yes/No step can present any content -- text, images, inputs, and so on -- in addition to its decision. It adds two required buttons that route execution down a **Yes** path or a **No** path. Configuration:

- Yes Value / No Value -- the values stored for each choice (default: "Yes" / "No")
- Default Selection -- pre-select Yes, No, or neither (require an explicit choice)

The node has two outgoing connections, one for the Yes branch and one for the No branch.

1.3.6 The User Interface Canvas

The **UI Canvas** is the screen designer used by both *User Interaction* and *Yes/No* steps. Open it by double-clicking the step. It is a what-you-see-is-what-you-get layout surface with three parts:

- Element palette (left) -- the elements you can add to the screen
- Canvas (center) -- the screen layout, shown inside a device frame
- Inspector (right) -- properties of the currently selected element

1.3.6.1 Device Layouts

Every screen is laid out for three device sizes, switched with the device selector. Positions and sizes are stored per layout in logical pixels and scaled to the real device at runtime:

- Phone -- 390 x 844
- Tablet -- 768 x 1024
- Desktop -- 1280 x 800

Lay out each screen for all three sizes so it reads well on phone, tablet, and desktop.

1.3.6.2 Common Element Properties

Every element has a *position and size* (X, Y, width, height) and a *stacking order* (Z-index). Add an element by dragging it from the palette or clicking it; select an element to edit its properties in the Inspector.

1.3.6.3 Element Types

Element	Purpose	Key Options
Text	A line or block of body text	Content, font size (default 16), weight (normal/bold), color, alignment (left/center/right)
Text Input	Single line input	Label, placeholder, field name, required, input mode (text, number, phone, password, dropdown, combo box), list items / list source (for dropdown & combo box), default & placeholder sources, *output parameter* (the Value Property that stores the entry)
Multi-line Text	Multiple line text area	Label, placeholder, field name, required, rows (default 4), font size, default & placeholder sources, *output parameter*
Checkbox Group	Multiple-selection list	Label, field name, options (label/value pairs), list source, required, font size, *output parameter*

Element	Purpose	Key Options
Radio Group	Single-selection list	Label, field name, options (label/value pairs), list source, required, font size, *output parameter*
Image	Display an image	Source (an uploaded image or an `https://` URL), object fit (contain/cover/fill)
Video	Embed a video	Source (`https://` URL), optional poster or thumbnail URL. NOTE: Video is not yet available on Android
Divider	A horizontal line	Thickness (default 1px), color
Timer	A countdown or count-up timer	label, field name, duration in seconds (default 300), direction (countdown or countup), block "Done" until finished, default source, *output parameter*

Capturing input: Elements that collect data (Text Input, Multi-Line Text, Checkbox Group, Radio Group, Timer) write their result to the *output parameter* you set -- a Value Property entry in `Property.Entry` dot-notation. List-driven elements dropdown and combo box inputs, checkbox and radio groups) can take their options from a static list or from an input parameter via *list source*. Default and placeholder values can be static text, an input parameter, or a Value Property reference.

1.3.7 Script Step

Executes TypeScript code. Features:

- CodeMirror 6 editor with TypeScript syntax highlighting and line numbers
- *Generate Template* button creates a commented template showing input/output parameter usage
- *NOTE: In the runtime system, you must turn on Settings option to "Allow SCRIPT step execution". It is turned ON by default..*

1.3.7.1 Input Parameters

- Each *input_parameter_specifications* entry becomes a local variable in the script, named by its *id*.
- If *value_type property*`, the variable's value is resolved from the property store at the dot-notation key in *default_value*.
- If `value_type: "literal"` (or omitted), the variable's value is the `default_value` string directly.
- All values are strings.

- If a property reference resolves to a group (no exact entry match), it returns a JSON array of `[{"name":"...","value":"..."}]`.

1.3.7.2 Output Parameters

- An `*output*` object is provided to the script automatically.
- Write results as `*output.someKey* = *value*`.
- If `*output_parameter_specifications*` exist: each spec's `*id*` is matched against output keys, and the value is written to the property store at the spec's `target` dot-notation key.
- If no output specs exist: output keys are written directly as dot-notation property keys.

1.3.7.3 Example

Given these specs on a SCRIPT step:

```
```json
{
  "input_parameter_specifications": [
    {"id": "name", "default_value": "User.Name", "value_type": "property"},
    {"id": "age", "default_value": "30"}
  ],
  "output_parameter_specifications": [
    {"id": "greeting", "target": "Result.Message"},
    {"id": "eligible", "target": "Result.Eligible"}
  ],
  "script_config": {
    "source":
    "output.greeting = name + ' is ' + age;
     output.eligible = Number(age) >= 18 ? 'yes' : 'no';"
  }
}
```

If `User.Name = "Alice"` in the property store, the script executes with `name = "Alice"`, `age = "30"`, and writes:

- `Result.Message` = `"Alice is 30"`
- `Result.Eligible` = `"yes"`

1.3.8 Child Workflow

Represents a sub-workflow. Two modes:

- **Embedded** -- A copy of the workflow stored within the parent. Double-click to navigate into it. Fully editable with its own inputs, outputs, values, and resources.
- **Reference** -- A link to an existing step library specification. Changes to the library spec are reflected wherever it is referenced. Properties are read-only.

1.3.9 Select 1

Conditional branching node. Routes execution to one of several paths based on comparing an input value against options.
```

Configuration:

- **Input Parameter Name** -- The value to compare (literal text or a value property reference)
- **Comparison Options** -- Ordered list of conditions:
- **Label** -- Display name for the branch
- **Operator** -- =, !=, <, >, <=, >=, contains, does not contain
- **Value** -- The comparison value (literal or property reference)
- **Default** -- One option can be marked as default (takes this path if no other condition matches)

Each option creates a corresponding output handle on the node. New connections from a Select 1 node automatically create a new option if none exists.

1.3.10 Parallel / Wait All

Use together for parallel execution:

1. **Parallel** splits into multiple branches
2. **Wait All** waits for all branches to complete before continuing

1.3.11 Wait Any

Used with **Select 1** or **Yes/No** step. Continues as soon as any one branch completes.

1.3.12 Catch

Used to start a CATCH workflow. Catch workflows are triggered through the use of a TRY check in an environment action definition. If the action fails (Aborts, Errors, or Times out) then the CATCH network is triggered. There can be multiple CATCH networks on the same diagram as the main workflow.

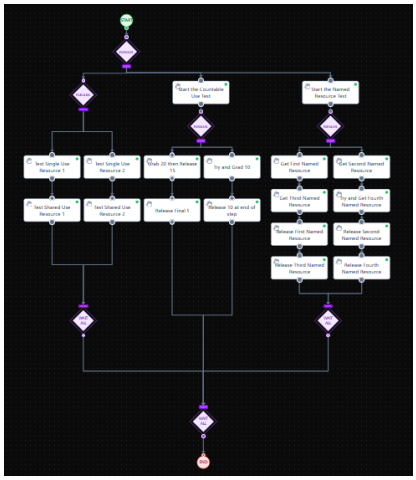
1.3.13 Return

Used to end a CATCH workflow. Returns have multiple options to abandon the workflow, restart the workflow, go to a specific step in the workflow, or retry the triggering step.

1.4 Notation Systems

Trajectory Workflow Editor supports three visual notation systems. All three represent the same workflow logic -- only the visual appearance and layout direction change.

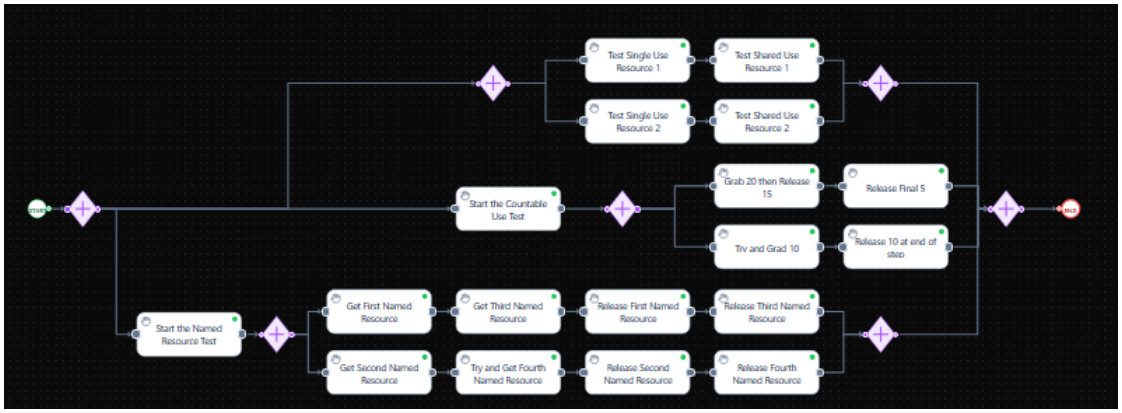
1.4.1 Flowchart Style (Default)



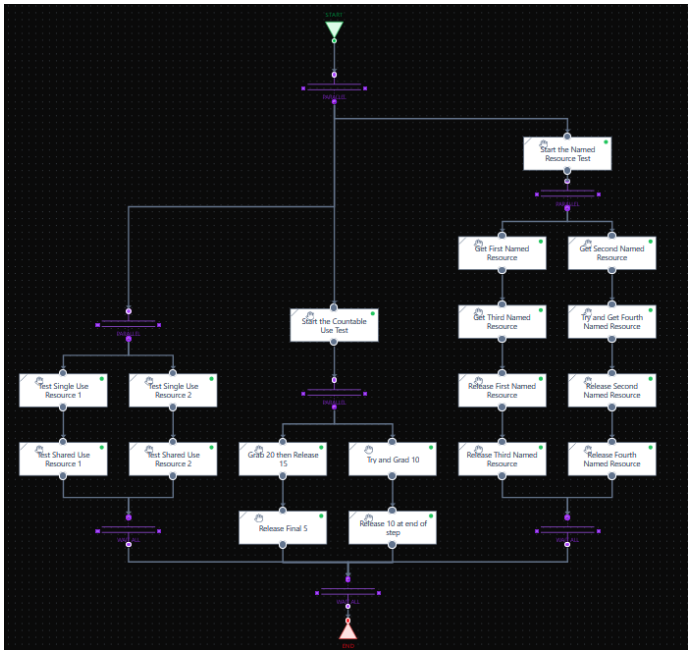
- Layout direction - Top to bottom
- Start/End - Filled circles (green/red)
- Actions and steps - Filled rectangles
- Select 1 / Wait Any - Filled diamonds (amber)
- **Parallel / Wait All - Nested diamonds (purple)

1.4.2 BPMN Style

- Layout direction - Left to right
- Start - Thin circle outline (green)
- End - Thick circle outline (red)
- Tasks - Rounded rectangles with step-type icons
- Exclusive Gateway (Select 1, Wait Any - Diamond with X marker (amber)
- Parallel Gateway (Parallel, Wait All -Diamond with + marker (purple)



1.4.3 ISA 88 Style (Procedural Function Chart)



- Layout direction - Top to bottom
- Start/End - Circle outlines (green/red)
- Steps:* - * Rectangles with corner hierarchy lines
- Selection (Select 1, Wait Any - Single horizontal bar (amber)
- Parallel (Parallel, Wait All - Double horizontal bar (purple)

1.4.4 Switching Notations

1. Click the notation dropdown in the toolbar (shows current notation, e.g., "Flowchart Style").
2. Select a different notation.
3. If the canvas has nodes, a confirmation dialog appears explaining the visual changes.
4. Click **Switch Notation** to confirm.
5. All nodes are automatically re-arranged to the new direction (left-to-right for BPMN, top-to-bottom for Flowchart and ISA 88).

Each workflow remembers its notation setting. Switching workflows restores the saved notation.

1.5 Environment and Action System

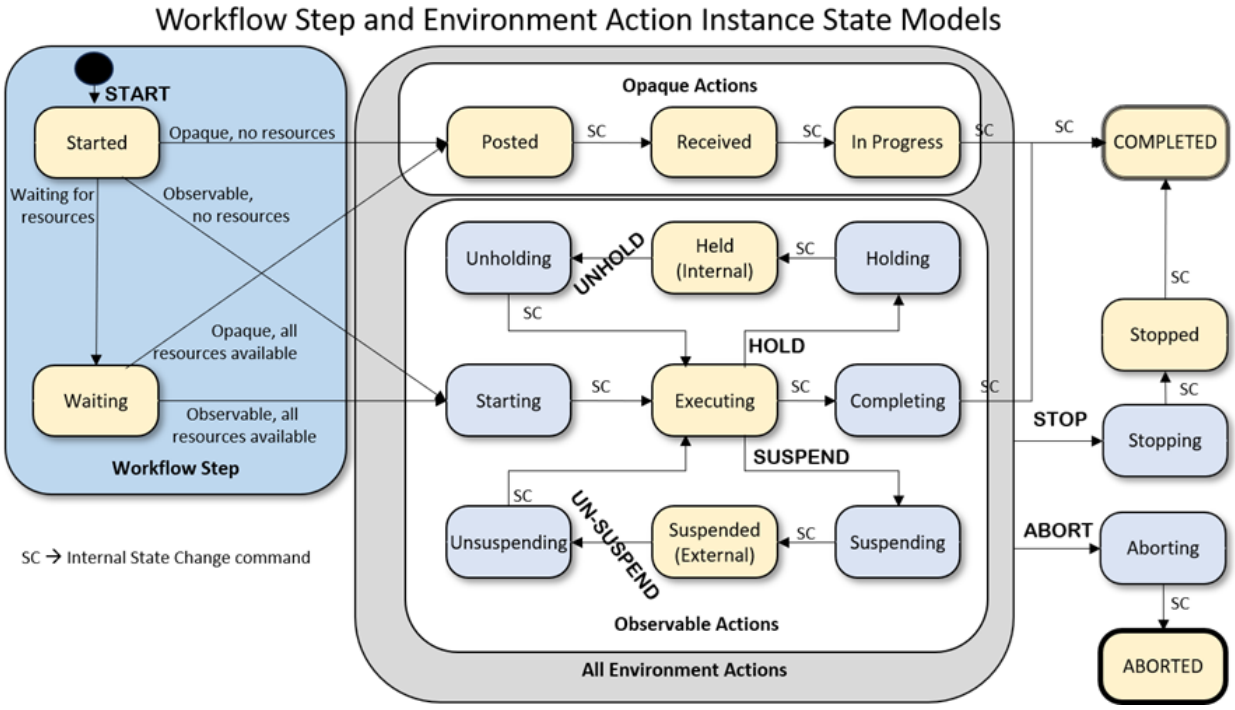
1.5.1 Concept

Actions define reusable operations (e.g., "Send Email", "Query Sensor") with untyped inputs and outputs, passed as strings. Actions are organized into Action Libraries to aid in managing large collections of actions. Action libraries are organizational only, individual actions are linked to Environments, and Environments are downloaded to the Action execution system (Trajectory Action Container).

- Environments group the actions available in a specific context (e.g., "Production Line", "Test Lab") and may also define value properties and resources.
- Workflows link to environments to gain access to their actions.

1.5.2 State Models

Each workflow step and each Environment Action instance steps through a defined lifecycle at runtime. The diagram below shows both side by side -- the workflow step lifecycle on the left, and the action instance lifecycle (opaque and observable variants) on the right.



- Workflow Step states (left panel) -- every step begins in **Started.** If the step is waiting on resources (an acquire that hasn't completed, a sync receive, a child workflow), it moves to **Waiting** and stays there until the resources become available. Once they are, the step proceeds into its Environment Action: observable actions enter at **Starting,** opaque actions enter at **Posted**
- **Opaque Action** states (top right group) -- the simplified lifecycle: **Posted -> Received -> In Progress -> COMPLETED**. The container drives the internal transitions automatically; the workflow only sees the start, the end, and any abort or stop. Opaque actions support **Abort** and **Stop**, but not **Hold** or **Suspend**.
- **Observable Action** states (bottom right group) -- the full ISA-88-inspired lifecycle: **Starting -> Executing -> Completing -> COMPLETED**, with code-driven **Hold/Unhold** through **Holding** -> **Held**(Internal) -> **Unholding**, and operator-driven **Suspend/Unsuspend** through **Suspending** -> **Suspended**(External) -> **Unsuspending**. Any active state can be **Stopped**

(clean shutdown to **COMPLETED** via **Stopping** -> **Stopped**) or **Aborted** (error shutdown to **ABORTED** via **Aborting**).

- Internal hold vs external pauses -- **Held** (Internal) is the action pausing itself: code running in **Executing** detects a condition it cannot proceed past (for example, a downstream queue is full) and asks the container to hold. **Suspended** (External) is an operator or the runtime pausing the action from outside. The two paths share the same shape but exist for different reasons; the Trajectory Action Container section includes a worked example of the internal Hold/Unhold cycle.
- **SC** (Internal State Change command) annotations on the diagram mark transitions the container drives automatically once a state's code finishes. Named labels -- **Hold**, **Unhold**, **Suspend**, **Unsuspend**, **Stop**, **Abort** -- are triggered either by the action's own code or by the runtime or operator.

The Workflow Editor displays each Environment Action as a single node on the canvas; the lifecycle shown above is what each action runs through inside the Trajectory Action Container after the workflow reaches it.

1.5.3 Linking Environments to a Workflow

1. Open a workflow specification on the canvas.
2. In the **Specification Header** above the canvas, click the environment dropdown.
3. Select an environment specification from the grouped list.
4. The environment appears as a removable badge. Its actions become available in the Step Palette.

To remove an environment, click the **X** on its badge.

1.5.4 Using Actions in a Workflow

After linking environments:

1. Drag an environment button from the Step Palette, or drag a generic "Env Action" step.
2. Double-click the node to edit properties.
3. Use the **Action Reference** dropdown to select a specific action.
4. The node's inputs and outputs auto-populate from the action specification.

1.5.5 Creating Action and Environment Specifications

1. Switch to the **Actions** or **Environments** tab in the Library Sidebar.
2. Create a library, then add specifications.
3. For actions: define inputs, outputs, properties, and visibility (opaque/observable).

4. For environments: the detail view uses a tabbed layout with five tabs:
    a. **Included Actions** -- actions available in this environment, referenced by library name and action name
    b. **Property Values** -- named value properties scoped to the environment
    c. **Action Properties** -- value properties specific to action execution
    d. **Resources** -- shared resources (Single Use, Shared Use, Count Limited, Named, Sync) that steps can acquire/release
    e. **Registered Action Servers** -- servers that implement the actions defined in the environment (see below)

1.5.6 Registered Action Servers

The **Registered Action Servers** tab in the Environment detail view is where you register the servers that perform the actions defined in the environment. Each entry identifies a server endpoint that the runtime can call to execute an Environment Action used by the workflow.

Each registered server has:

1. **Name** -- a display name identifying the server (e.g., "Production Line Controller")
2. **URI** -- the endpoint address (e.g., `https://server-address:port`)
3. **API Key** – The API Key to access the URL Rest point (optional)
4. **Description** -- optional notes about the server's purpose or capabilities
5. **Connection Type** -- the interface protocol used to communicate with the server. Currently only **REST** is supported; other interface types may be added in future versions.

At runtime, when a workflow step executes an Environment Action, the engine consults the list of Registered Action Servers defined in the environment to identify which server(s) can implement that action.

1.6 TRY, CATCH, and RETURN

The TRY option on an Environment Action provides a way to attempt recovery from the failure of an Environment Action. There can be up to three TRY statements per Environment Action. The TRY statement identifies what CATCH workflow to use if the action fails. The TRY statement may specify: TRY on ABORT, TRY on TIME OUT, and TRY on Error.

The CATCH node is the start of a workflow that is triggered by a failure of an action. Each CATCH in a workflow diagram must have a unique identifier/

The RETURN node is the end of a CATCH workflow. The RETURN may specify that: ABANDON the workflow, RESTART the workflow, RETRY the trigger step, or GO TO a specific step in the workflow (which must be in the same execution chain as the trigger step).

There is an option on the CATCH to release all resources before the CATCH starts, in order to not lockup a resource.

1.7 Resources

Resources model shared physical or logical assets that workflow steps must coordinate access to. Resources are defined at the workflow or environment level and consumed by steps using resource commands.

1.7.1 Resource Types

Type	Description	Count	Properties
Single Use	Exclusive access to a single resource. Only one step can hold it at a time.	1	Name, Description
Shared Use	Shared access with pool limits. Multiple steps may hold it concurrently.	User Defined	Name, Description, Count
Count Limited	A pool of identical units. Steps acquire and release a specified number of units.	User Defined	Name, Description, Count
Named	A pool of individually named items. The acquired item's name is stored in a value property.	Number of items in the list	Name, Description, Name list
Sync	A synchronization point for passing data between parallel branches via Send/Receive.	2 Sync points per SYNC	Name, Description, Sync Type

1.7.2 Resources Scope

Resources can be defined at two levels:

- **Workflow level** -- In the Properties Panel when no node is selected, under the **Resources** section. These are available to all steps in the workflow and its embedded child workflows.
- **Environment level** -- In an Environment specification's resource section. These become available to any workflow that links to the environment.

1.7.3 Resource Definition

To add a resource, click the **+** button and configure:

1. **Name** (required) -- Unique identifier for the resource. Spaces are automatically removed.
2. **Type** -- Select from the five resource types.
3. **Description** (optional) -- Human-readable explanation.
4. **Count** -- Editable for Count Limited type; auto-calculated for Named type; fixed at 1 for others.
5. **Names** (Named type only) -- A list of individual item names. Add or remove names with the **+** and trash buttons.

1.7.4 Resource Commands

Steps consume resources through resource commands, configured in the step's Properties Panel. Commands are organized into three sections:

1.7.4.1 Acquire at Step Start

Claim a resource before the step executes.

Field	When Shown	Description
Resource	Always	Select from available resources (grouped by source: Workflow, Environment, Parent).
Count	Count limited only	Number of units to acquire.
Store name to	Named Only	Value property where the acquired item's name is stored.

1.7.4.2 Release at Step End

Release a previously acquired resource after the step completes.

Field	When Shown	Description
Resource	Always	Select the resource to release
Count	Count limited only	Number of units to release
Release Resource Stored In	Named Only	Value property that contains the named resource to be returned to the pool

1.7.4.3 Sync Command

Coordinate data exchange between parallel branches. Only one sync command is allowed per step.

Command Type	Description	Additional Fields
Send	Send a value through the sync resource	**Parameter to send** -- input parameter or value property reference
Receive	Receive a value from the sync resource	**Store received value to** -- output parameter or value property
Synchronize	Synchronize execution without transferring data	None

1.7.5 Execution Order

Resource commands that return values execute **before** input parameters are sent to the step. This applies to:

- **Named Resource Acquire** -- The acquired item's name is stored in a value property before the step runs, so it can be referenced as a step input parameter.
- **Sync Receive** -- The received data is stored in a value property before the step runs, so it can be used as a step input parameter.

This ordering allows a step to use the acquired name or received data as part of its input, for example passing the name of an acquired machine to an action that operates on that specific machine.

1.7.6 Resource Scope and Inheritance

Steps can use resources from multiple sources, listed in priority order:

1. Current workflow's own resources
2. Resources from environments linked to the current workflow
3. Parent workflow's resources (if the current workflow is an embedded child)
4. Resources from environments linked to parent workflows

The resource selector in the command editor groups available resources by source (e.g., "Workflow Resources", "Env: Production Line", "Parent: Main Process"). Each resource can only be acquired or released once per step.

1.8 Child Workflows

1.8.1 Creating a Child Workflow

1. Drag the "Child Workflow" step from the palette onto the canvas.
2. A creation dialog appears with three modes:

Mode	Description
Create New	Creates a new empty embedded sub-workflow.
Copy from Library	Copies the contents of an existing step library specification into a new embedded sub-workflow.
Reference	Links to an existing step library specification (changes to the library spec are shared across all references.

3. Choose a mode, configure the options, and click **Create**.

1.8.2 Navigating Child Workflows

- **Double-click** an embedded child workflow node to navigate into its canvas.
- A **Back** button appears in the toolbar to return to the parent workflow.
- Embedded child workflows inherit their parent's environment links.

1.8.3 Embedded vs. Referenced

Feature	Embedded	Referenced
Editable content	Yes	No (read-only)
Navigate into canvas	Yes (Double click)	No
Shares Changes	No (Independent copy)	Yes (all references update)
Source	Created in-place	Links to Step Library

1.9 Export and Import Formats

Format	Extension	Description
Workflow Package	.WFmasterX	Self-contained runtime ZIP with workflow, environments, actions, and images. Primarily for use with the Runtime, but may also be imported into the editor. NOTE: When importing to the editor, any referenced child workflows are converted to local copies.
Workflow Library	.WFlibX	Library ZIP bundle of all workflows in a library with all dependent step, environment, and action libraries.
Environment Library	.WFenvirLibX	Library ZIP bundling one or more environments and their included actions
Action Library	.WFactionLibX	Library ZIP bundling one or more action libraries
Step Workflow Library	.WFstepLibX	Library ZIP bundle containing all step workflows in the library and associated environments and actions.

Single Environment Package	.WFenvirX	ZIP containing a single environment spec plus the actions it includes (round-trips into one environment)

Legacy extensions (`.WFenvir`, `.WFaction`, `.WFslib`, `.WFactionX`) are still accepted on import for backward compatibility.

1.9.1 Exporting

From the canvas toolbar (top-right):

- Export Workflow Package -- Creates a .WFmasterX runtime package from the current workflow.

From the library sidebar export menu:

- Select a library, then click the download icon.
- For workflow libraries: creates a .WFlibX package bundling all dependencies.
- For other library types: creates a JSON library export.

1.9.2 Importing

From the library sidebar import button (upload icon):

- Click the upload icon and select a file matching the current entity type.
- For workflow libraries: accepts both .WFlibX ZIP packages and plain JSON files.
- ZIP packages import all bundled dependencies (actions, environments, steps) automatically.
- Duplicate libraries are detected by name and entity type and skipped on import.

From the library context menu:

- Right-click a workflow library and select **Import Workflow Package** to import a .WFmasterX file.
- The imported package's actions and environments are created first, then the workflow is loaded.

1.9.3 What's Inside a Workflow Package (.WFmasterX)

workflow.WFmaster	Workflow JSON (nodes, edges, properties)
environments/name.WFenvir	Environment library JSON files
actions/name.WFaction	Action library JSON files
images/filename.png	Referenced image files
manifest.json	Package metadata

Use this format to export to the runtime. It may also be used to reimport back to an editor. However, any child workflows used by reference are converted into local copies.

1.9.4 What's Inside a Library Package (.WFlibX)

library.WFlibX	Workflow library JSON
steps/name.WFslibX	Step library JSON files
environments/name.WFenvir	Environment library JSON files
actions/name.WFaction	Action library JSON files
images/filename.png	Referenced image files
manifest.json	Package metadata

1.9.5 What's Inside a Single-Environment Package (.WFenvirX)

{EnvName}.WFenvir	The environment spec JSON (no library wrapper)
actions/{LibName}/*.WFaction	One file per included action
manifest.json	packageType: "environment-package"

Use this format when you want to share a single environment together with the actions it references. Import treats it as a self-contained unit and creates exactly one environment.

1.9.6 Schema Version

All exports use schema version 4.0. The schema uses snake_case field names in exported JSON (converted from camelCase internal representation).

1.9.7 Identifiers

Trajectory Workflow Editor uses Snowflake OIDs (Ordered Identifiers) -- 64-bit IDs that encode a timestamp, entity type, and sequence number. These are generated client-side.

1.10 Version and State Management

Every specification (workflow, step, environment, or action) carries a **version** and a **lifecycle state**, and the editor keeps a history of both.

1.10.1 Versions

- Each specification has a semantic version (e.g., `1.0.0`), shown on a version badge.
- Bumping the version records a history entry that links back to the previous version, so you can trace how a specification evolved.

- The version history records creation, version bumps, and state changes -- each with the account and timestamp.

1.10.2 Lifecycle States

A specification moves through a defined set of states as it matures. These are for information purposes only, there are no internal checks or locks on different states.

State	Meaning
Draft	Work in progress, under authorship
In Test	Being tested, not yet ready for review
In Review	Waiting for approval
Approved	Approved after review, not yet effective
Effective	Published and active for use
Superseded	Replaced by a newer version
Obsolete	No longer valid

Each state change is recorded in the specification's history.

1.11 Other User Interaction

1.11.1Saving

The editor **saves automatically** as you work. Edits are debounced and flushed to the server, and the toolbar's **Save** status indicator shows whether changes are pending or saved (and offers a retry if a save fails).

In addition, your in-progress canvas is mirrored to the browser so it can be recovered after an unexpected close.

1.11.2 Editing Keyboard Shortcuts

Shortcut	Action
Ctrl+C and Cmd+C	Copy selected nodes and their inter-connections
Ctrl+V and Cmd+V	Paste copied nodes (offset 20px from originals, with new IDs)
Delete and Backspace	Delete selected nodes and edges
Ctrl+Z and Cmd+Z	Undo (up to 100 states)
Ctrl+Shift+Z and Cmd+Shift+Z	Redo
Ctrl+Y and Cmd+Y	Redo (Windows alternative)

1.11.3 Selection Buttons

Shortcut	Action
Click	Select a single node or edge

Shift+Click and Ctrl+Click	Add to or toggle selection
Shift+Click and Ctrl+Click	Add to or toggle selection

Keyboard shortcuts are disabled when typing in text fields, textareas, or editable labels to prevent interference with normal text editing.

1.11.4 Draft Recovery

Trajectory Workflow Editor automatically saves your canvas state to the browser's IndexedDB storage as you work. This includes nodes, edges, workflow properties, and images.

1.11.5 Recovery on App Load

If a draft is detected when you open the application, a dialog appears

```
Recover unsaved work?
Found an unsaved draft from [timestamp]. Would you like to continue
where you left off?
```

- Recover Draft – Loads the saved state and clears the undo history
- Start Fresh – Discards the draft and starts with an empty canvas

1.11.6 When Drafts Are Cleared

- When you choose "Start Fresh" from the recovery dialog
- When you select and load a workflow specification from the library
- When you sign out

1.12 User Accounts and Roles

1.12.1 Role System

Each user has a set of author roles that control which library types they can create and edit:

Role	Permission
Workflow Author	Create and edit workflow libraries and specifications
Step Library Author	Create and edit step libraries and specifications
Environment Author	Create and edit environment libraries and specifications
Action Author	Create and edit action libraries and specifications
Administrator	Manage all user accounts and roles. The first registered user is automatically promoted to administrator.
Global	Edit shared libraries owned by other users. Required to toggle library sharing.

1.12.2 Ownership and Sharing

- You can always edit libraries you own.
- **Shared libraries** (marked with a globe icon) are visible to all users.
- Only Global users can edit shared libraries they don't own.
- Only Global users can share or un-share a library.

1.12.3 Account Management

Administrators can manage user accounts by clicking the user icon in the top-right corner. From the Account Management dialog, administrators can:

- View all registered users
- Assign or remove author roles
- Grant or revoke administrator and global privileges

2 Trajectory Runtime

***PLEASE NOTE**: Trajectory is a demonstration system, not intended for production environments. The editor and runtime are single-user systems and do not have the security necessary for production use. We recommend loading the applications into a Docker container for your testing.*

2.1 Executing a Workflow

The editor produces workflow specifications; the Trajectory runtime applications execute them. The bridge between them is the **Workflow Package** (`.WFmasterX`).

2.2 Export the workflow package

With a workflow open, use the **Export** dropdown in the top-right of the workflow editor pane and choose **Export Workflow Package**.

This produces a single `.WFmasterX` file -- a self-contained ZIP that bundles the workflow, every environment and action library it depends on, and all referenced images, so it runs without any other files.

(The same dropdown also offers PDF, PNG, and SVG exports for documentation and diagrams.)

2.3 Run it on an Android phone or tablet

A native Trajectory runtime app for Android renders the same workflows, including the UI Canvas screens you designed. Because this is a demonstration system, it is not distributed through an app store -- you build and install it yourself:

1. **Enable Developer Options** on the device: open **Settings** -> **About phone** and tap **Build number** seven times.
2. **Turn on USB debugging**: Settings -> System -> Developer options -> USB debugging.
3. **Install the app over USB**. Connect the device to your computer and accept the debugging prompt on the device. Then either open the Android project (`engines/android-app`) in Android Studio and Run it onto the device, or install a prebuilt APK with `adb install <app>.apk`.

Getting a `.WFmasterX` onto the device. A few options:

- **File transfer** -- copy the `.WFmasterX` to the device over the USB connection (or with `adb push`), then load it from within the app.

- **Cloud or email** -- send the file to the device (Drive, email, etc.) and open/import it in the app.
- **Download** -- host the `.WFmasterX` somewhere reachable and download it on the device, such as a Google drive.

Once the file is on the device, load it from within the app the same way as the web runtime.

2.4 Run it in the web runtime (Trajectory Runtime)

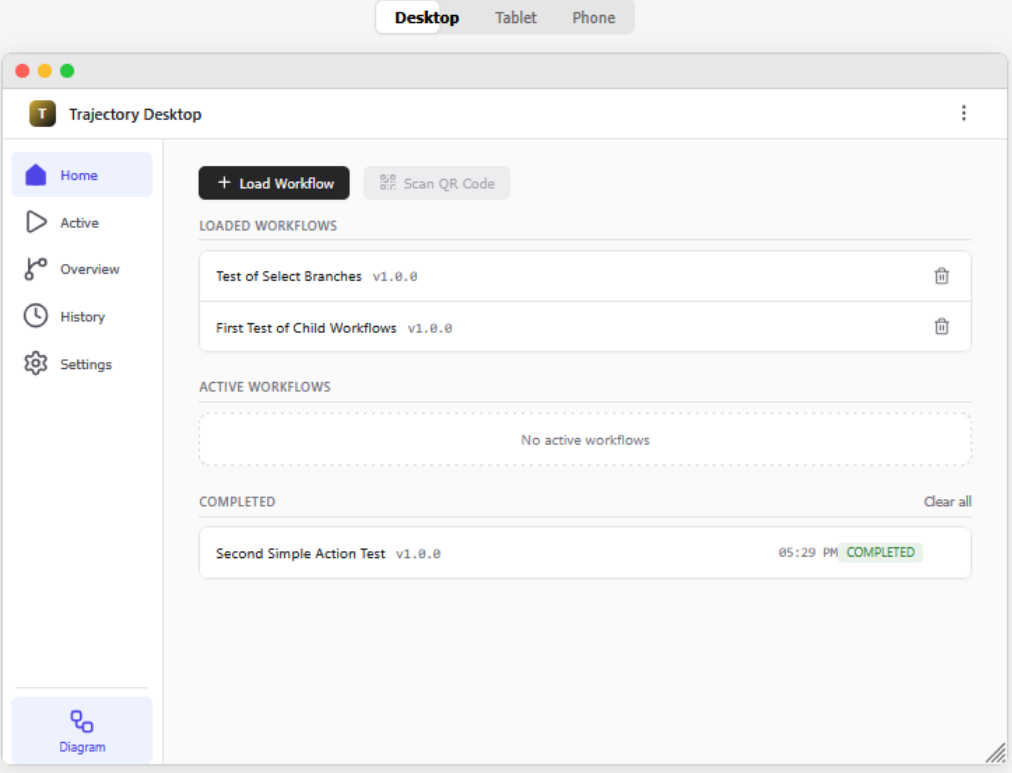
Trajectory Desktop is the **workflow runner** for the Trajectory family. It loads workflow packages (`.WFmasterX`) created in the Trajectory Workflow Editor and runs them step by step — entirely in your browser.

The simplest way to run a workflow -- no installation required:

1. Open the Trajectory Runtime web UI in a browser.
2. On the Home screen, use the **Load Workflow** control (the upload button) and pick your `.WFmasterX` file, or -- on desktop -- drag the file onto the drop zone.
3. The workflow loads; start it and step through it in the browser.
4. No sign-in is required, and nothing is installed — workflows run in the browser.

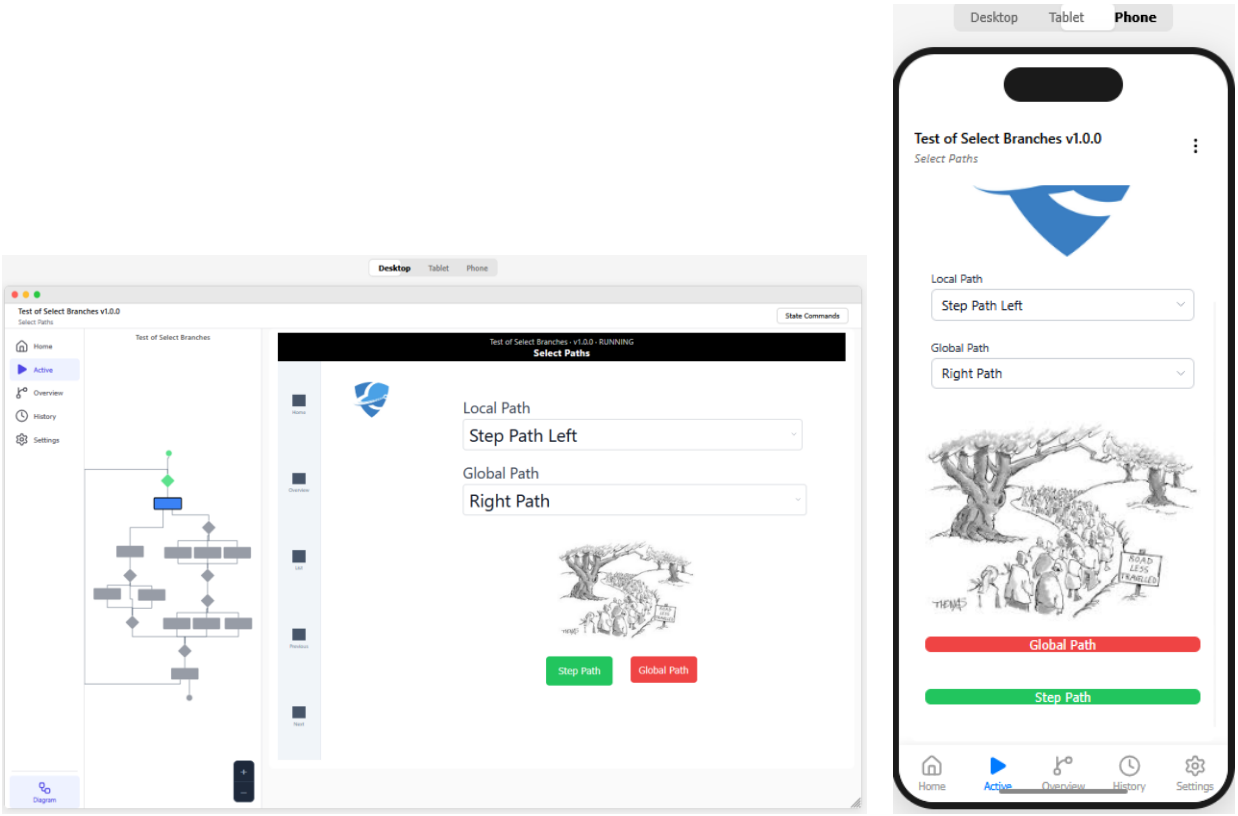
Workflow packages are loaded and executed entirely in the browser; the web runtime needs no backend. (Environment Actions are the exception -- those call out to a Trajectory Action Container.)

NOTE: If the downloaded workflow has a SCRIPT step, you must first turn on Settings option to “Allow SCRIPT step execution”. It is turned ON by default..



Open Trajectory Desktop in your browser. The app has five screens, reached from the tab bar (along the bottom on a phone, down the left side on a tablet or desktop):

Screen	Purpose
Home	Load workflow packages and start them
Active	Step through the workflows that are currently running
Overview	See live diagrams of running workflows
History	Review workflows that have finished
Settings	Preferences, data management, and this help guide



2.4.1 Loading a Workflow

On the **Home** screen:

1. Click **Load Workflow** and choose a `.WFmasterX` file (the package exported from the Trajectory Workflow Editor). On desktop you can also **drag and drop** the file onto the drop zone.
2. The workflow appears under **Loaded Workflows**.
3. Click it to open the **Start** dialog, which shows the workflow name, version, and description.
4. Click **Start** to begin. You are taken to the **Active** screen.

A workflow can require a connection to an Action Container before it starts, if the workflow uses an Environment Action. If there are multiple registered action servers for the environment, then a dialog asks you to select the action server to connect to.

2.4.2 Running a Workflow

The **Active** screen shows the steps that are currently running, one card at a time (swipe or use the dots to move between several active steps).

Interactive steps — **User Interaction** and **Yes/No** steps display the screen that was designed on the editor's UI Canvas.

Fill in any required fields; action buttons stay disabled until the required fields are satisfied. A **Yes/No** step shows its content plus two buttons that send the run down the Yes path or the No path.

A timer counts down (or up) and has play/pause and reset controls.

Automatic steps — such as scripts and waits — run on their own and show a brief "Processing…" card.

When a workflow finishes, a completion notice appears and the run moves to **History**.

2.4.3 Environment Actions and Action Servers

Some workflow steps are **Environment Actions** — operations that run on a separate **Trajectory Action Container** rather than in the browser. When you start a workflow that uses them, Trajectory Desktop asks which server to use:

- If the workflow's environment has **no registered server**, enter the Action Container's address (for example `http://localhost:3002`) and click **Connect**.
- If it has **several registered servers**, pick one and click **Use**.
- Click **Abandon workflow** to cancel instead.

Once connected, the workflow starts and its Environment Action steps call that server as they execute.

2.4.4 Overview and History

Overview shows a live diagram of each running workflow — in Flowchart, BPMN, or ISA 88 notation — with the current step highlighted. Use the dots to move between workflows.

History lists finished workflows with their outcome (Completed, Errored, or Stopped) and finish time. Open one to see a step-by-step log; click a step to view its input and output values (and any error or script source).

2.4.5 Settings

The **Settings** screen includes:

- **Notifications** — be notified when new steps become active (asks for browser permission).
- **Confirmations** — whether deleting loaded or completed workflows asks first.

- **Data Management** — clear all history, and view or release the Environment and Workflow resources currently held.
- **Action Server Mapping** — how a workflow's actions are matched to a server: **Name** (match by environment/action names) or **Exact** (the workflow's identifiers must match the server exactly).
- **About** — the app version and a link to this help guide.

3 Trajectory Action Container

The Action Container stores and runs the Actions that Trajectory workflows call. Actions are written in Python and run in a sandbox. This console is the web UI for managing containers, importing environments, writing and testing action code, and monitoring executions.

3.1 Action Getting Started

***PLEASE NOTE**: Trajectory is a demonstration system, not intended for production environments. The editor and runtime are single-user systems and do not have the security necessary for production use. We recommend loading the applications into a Docker container for your testing.*

The console has a top navigation bar and a left activity bar:

- Top bar
  - Dashboard (system overview)
  - Log (execution history)
  - Settings (configuration).
- Activity bar (left icons)
  - Explorer (browse environments and actions)
  - Instances (running executions)
  - Search

Start on the Dashboard for an at-a-glance view, then use the Explorer to drill into your environments and actions.

3.2 Environments and Actions

The container is organized into Environments, each bundling a set of Actions. These are the same environments a workflow links to in the Trajectory Workflow Editor.

- Import
  - On the Environments page click Upload Package and choose a `.WFenvirX` (zipped), `.WFenvir`, or `.WFaction` file. The import report lists what was created or updated.
- Browse
  - The Explorer panel shows a tree of environments and their actions.
- Environment detail lists each action with its visibility (observable or opaque) and

3.3 Writing Action Code

Open the Code Editor and choose an Environment → Action → State, then write Python in the editor.

An action's behavior is defined per state of its lifecycle:

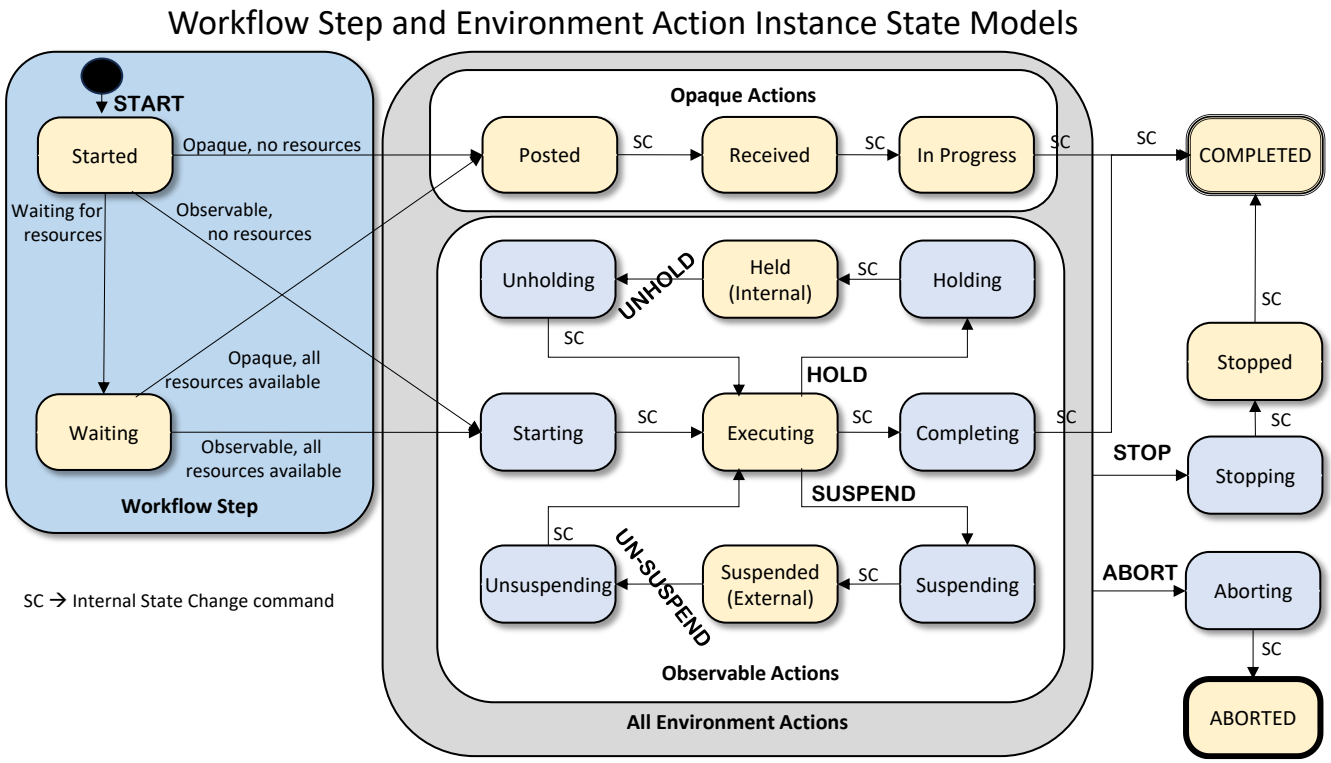
Observable – actions expose many states (STARTING, EXECUTING, COMPLETING, …)

Opaque – actions expose a few (POSTED, RECEIVED, IN_PROGRESS, …).

Code-status badges show which states already have code.

- Save Version stores a snapshot of the code, with an optional description.
- Version History lets you view earlier versions (read-only) and restore one to keep editing.
- On an action's detail page, you can Export Code / Import Code (`.WFactionCode`) and Clear Code for a state (irreversible).

3.3.1 Step and Action State Model



3.3.2 Coordinating state transitions from code

The Python code in each state can do more than process inputs and write outputs — it can also signal the next state transition. Three signals are available without any imports:

- Return `True` (or any non-`False` value, or no return at all) — the state's work is done; the container advances to the next state in the lifecycle (the SC transition).
- Return `False` from EXECUTING — code-initiated HOLD: the container moves through HOLDING into HELD, and waits there until an Unhold signal arrives.
- Raise any exception — code-initiated ABORT: the error summary and traceback are recorded on the instance and the container moves through ABORTING into ABORTED.

In addition, code in any active state can call `**trajectory.request_command("...")**` to request one of these commands after the current execution returns:

Command	Effect	Valid from
HOLD	Hold the action (the same destination as returning 'False').	Any Active State
UNHOLD	Release a held action, next state EXECUTING.	HELD only
PAUSE	Pause the action so an operator can resume it later.	EXECUTING only
STOP	Stop the action cleanly, next state COMPLETED.	Any Active State
ABORT	Abort the action without an exception, next state ABORTED.	Any Active State

A requested command that isn't valid for the current state is reported as an execution error and the action transitions to ABORTING.

3.3.3 Code Examples

Example: holding for a downstream queue

> A common reason to hold an action is to throttle against a downstream resource. A packer station, for example, may need to stop feeding a downstream wrapper whose buffer is full, then resume once the wrapper has caught up. The action defines code in four states; the container drives the cycle EXECUTING → HOLDING → HELD → UNHOLDING → EXECUTING and repeats it every time EXECUTING returns `False`.

EXECUTING — runs the normal operation; if downstream is full, ask to hold:

```python
def execute(inputs, outputs, props, action_props):
    if downstream_queue_full(props):
        return False          # code-initiated HOLD
    feed_one_item(inputs, props)
    return True      # SC -> COMPLETING when the batch is done
```

HOLDING — runs once on the way into HELD; bring the machine to a safe stopped state:

```python
def execute(inputs, outputs, props, action_props):
    stop_feed_motor(props)
    wait_until_machine_idle(props)
    return True                    # SC -> HELD
```

HELD — polls for the downstream queue to drain, then asks to unhold:

```python
def execute(inputs, outputs, props, action_props):
    wait_until_downstream_available(props)
    trajectory.request_command("UNHOLD")
    return True    # container moves HELD -> UNHOLDING
```

UNHOLDING — runs once on the way back to EXECUTING; restart the machine:

```python
def execute(inputs, outputs, props, action_props):
    start_feed_motor(props)
    return True
    # SC -> EXECUTING; the EXECUTING code re-checks the queue
```

ABORTING – runs if an unrecoverable error occurs in any state:

```
```python
def execute(inputs, outputs, props, action_props):
    start_feed_motor(props)
    if feed_motor-fails (props):
        trajectory.request_command("ABORT")
    return True
          # SC or ABORT code re-checks the queue
```
```

An operator can still take over from outside: the Instances page exposes Unhold (to release the hold manually), Abort, and Stop, all of which override what the action's own code is doing.

3.4 Testing Code

Click Test Code to open the test panel, fill in the action's input parameters, and click Run Test. The result shows a success/failure badge, execution time, the return value, an outputs table, and the captured stdout / stderr — a dry run without needing a workflow.

3.5 Instances

The Instances page lists executions in two tabs — Active (running) and History (finished). Open an instance to see:

- its current state with a color indicator,
- a state timeline of transitions and how long each took,
- the input and output parameters, and
- the pinned code versions the run used.

For observable actions, the detail view offers state-appropriate commands: Pause, Resume, Abort, Stop, Unhold.

3.6 Execution Log

The Log page records completed executions. Filter by action, environment, status (Completed / Aborted / Stopped), and date range; click a row to expand its inputs, outputs, and the states it executed.

3.7 Settings

The Settings page configures the container:

- Max Log Entries — how many execution records to keep.
- Python Pool Size — number of sandbox worker processes.

- Execution Timeout — per-state time limit in seconds (0 disables it).
- Instance Retention — hours to keep completed instance records.

Use Save Changes to apply, or Reset to Defaults. The page also shows read-only container info: version, uptime, and the current Python worker pool.

3.8 Connecting a Workflow to the Container

Workflow steps called Environment Actions run here, not in the runtime. To wire a workflow to this container, point it at the container's REST address — for example `http://localhost:3002`:

- In the Trajectory Workflow Editor, register the address as an action server on the workflow's environment, or
- in the Trajectory runtime, enter the address when prompted (the Action Server picker) as the workflow starts.

When a workflow reaches an Environment Action, the runtime calls this container, which runs the Python code you wrote and returns the outputs.